Microsoft

The Current State of JIT in PostgreSQL

Daniel Gustafsson

PGConf.EU 2026 - Valencia, Spain



Previously

vmware™ Pivotal curl://

https://archive.org/details/office-43

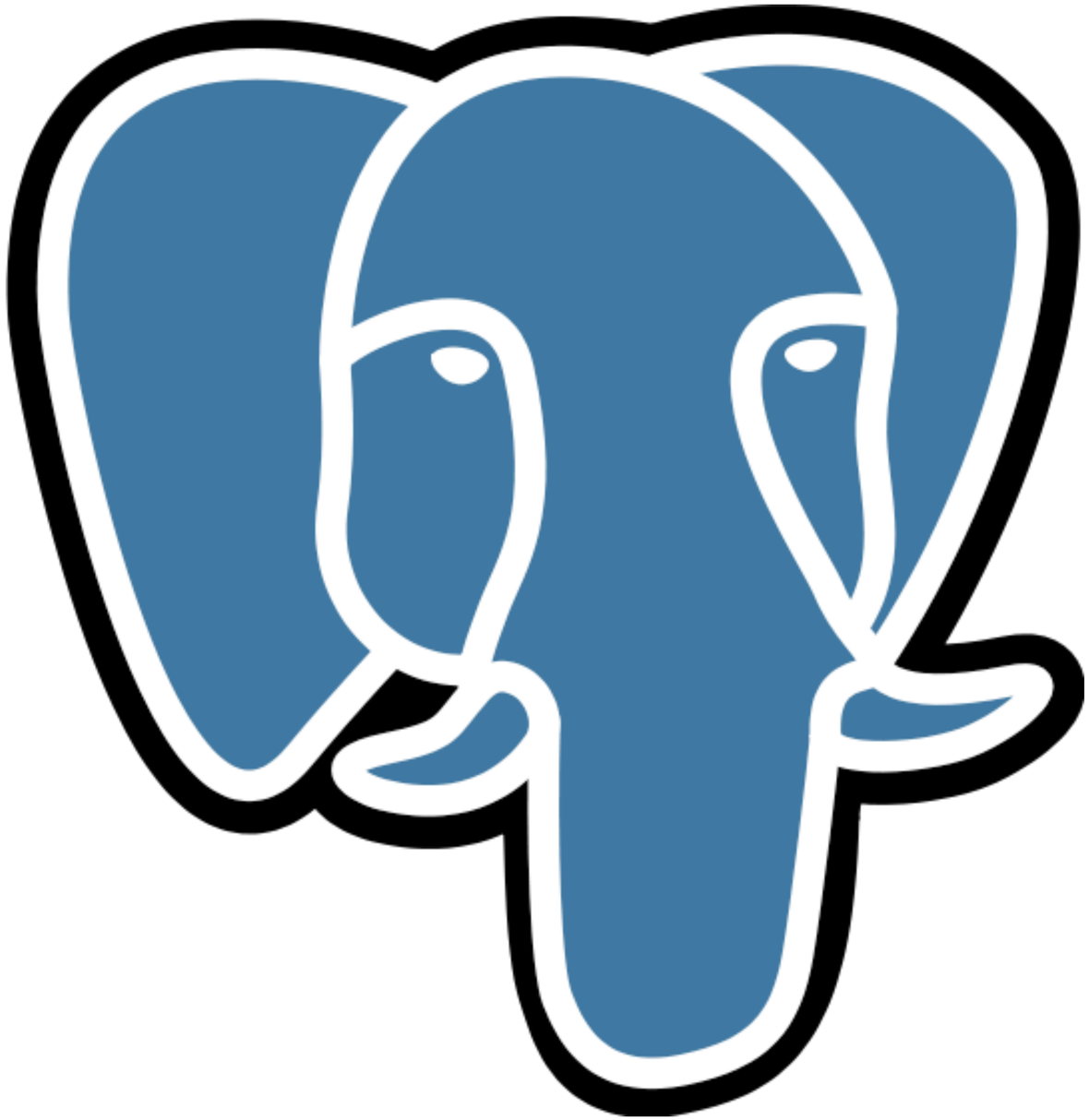


https://archive.org/details/office-43

User since March 5, 2005 (#blamebruce)

Contributing since ~ 2013

Major Contributor, Committer and Conference organizer



19

```
4685 files changed,
724534 insertions(+),
425038 deletions(-)
```

```
- #jit = on
+ #jit = off
```



JiT

| dʒɪt |

Just In Time Compilation



JuST IN *TIME*

| dʒɪt |

Just In Time Compilation

Faces of Open Source
Peter Adams

Ken Thompson



Regular Expression Search Algorithm

KEN THOMPSON
Bell Telephone Laboratories, Inc., Murray Hill, New Jersey

A method for locating specific character strings embedded in character text is described and an implementation of this method in the form of a compiler is discussed. The compiler accepts a regular expression as source language and produces an IBM 7094 program as object language. The object program then accepts the text to be searched as input and produces a signal every time an embedded string in the text matches the given regular expression. Examples, problems, and solutions are also presented.

KEY WORDS AND PHRASES: search, match, regular expression
CR CATEGORIES: 3.74, 4.49, 5.32



Faces of Open Source
Peter Adams

Regular Expression Search Algorithm

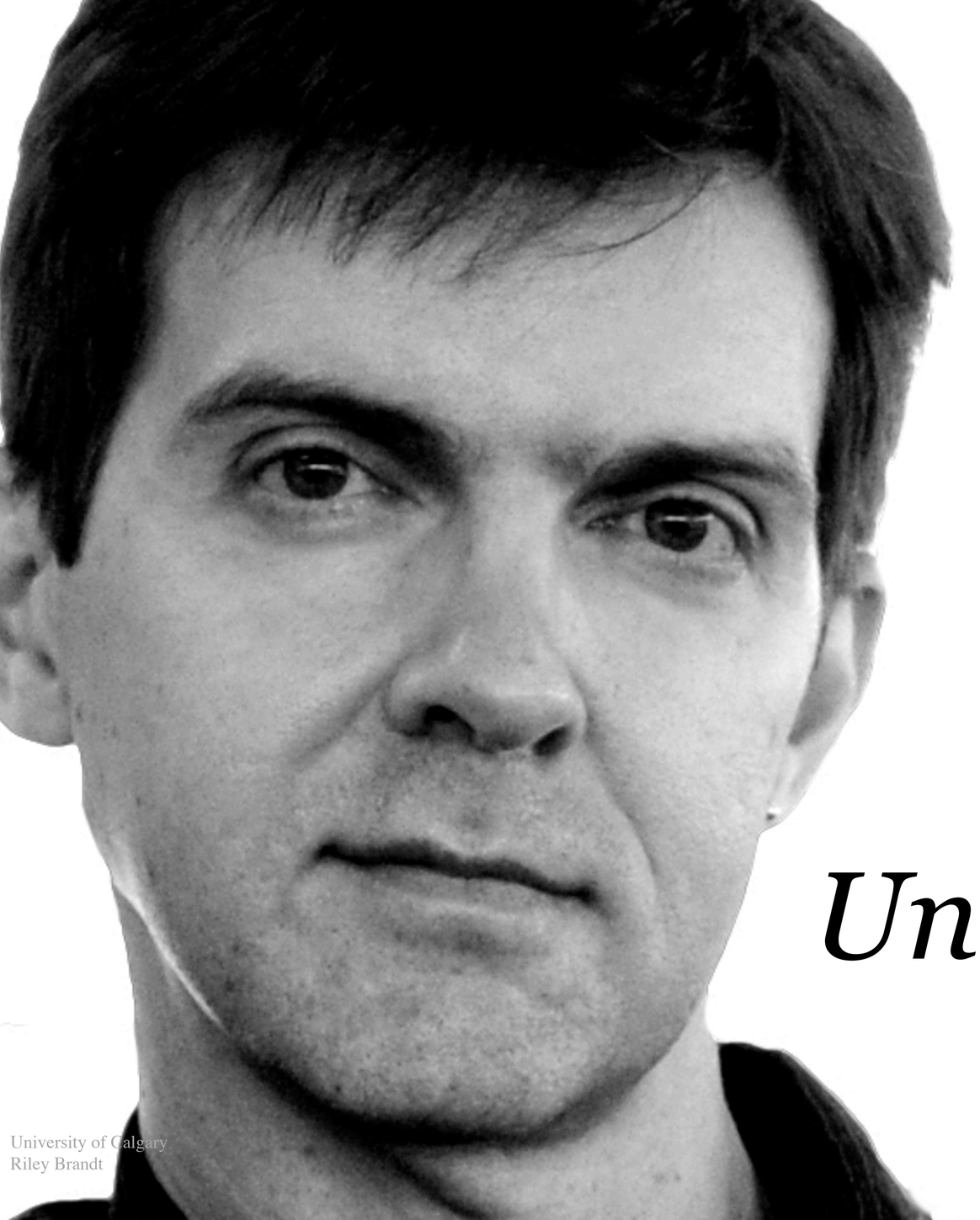
KEN THOMPSON
Bell Telephone Laboratories, Inc., Murray Hill, New Jersey

A method for locating specific character strings embedded in character text is described and an implementation of this method in the form of a compiler is discussed. The compiler accepts a regular expression as source language and produces an IBM 7094 program as object language. The object program then accepts the text to be searched as input and produces a signal every time an embedded string in the text matches the given regular expression. Examples, problems, and solutions are also presented.

KEY WORDS AND PHRASES: search, match, regular expression
CR CATEGORIES: 3.74, 4.49, 5.32



Faces of Open Source
Peter Adams



Professor

John Aycock

University of Calgary

> *"Strictly speaking, JIT compilation systems are completely unnecessary.*

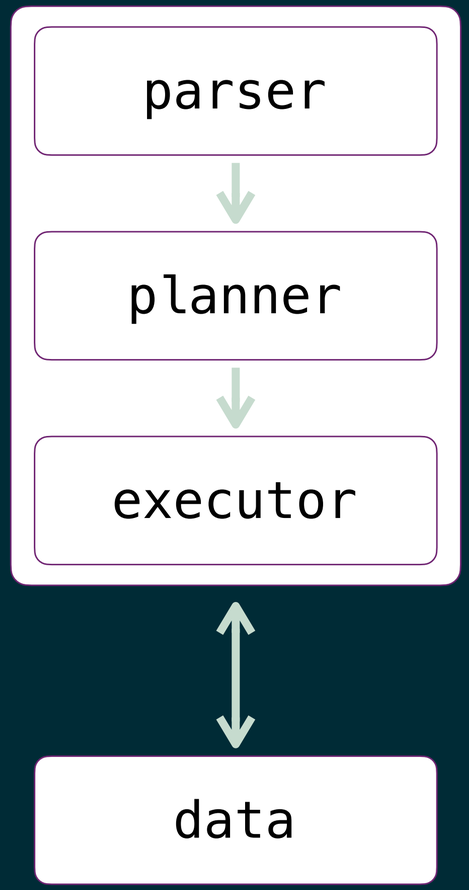
- Prof. John Aycock (A Brief History of Just-in-Time)

> *"Strictly speaking, JIT compilation systems are completely unnecessary.*
>
> *They are only a means to improve the time and space efficiency of programs."*

- Prof. John Aycock (A Brief History of Just-in-Time)

```
SELECT lorem FROM ipsum;
```

`SELECT lorem FROM ipsum;`



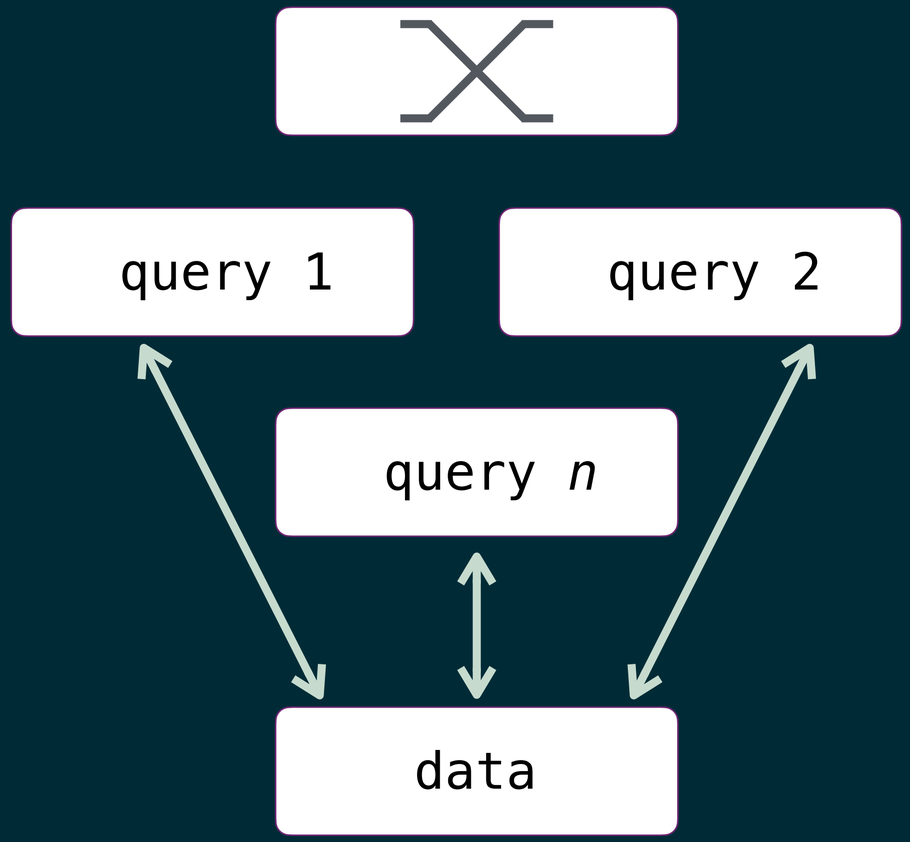
Generic program able to parse, plan and execute any syntactically valid SQL query

Space efficient

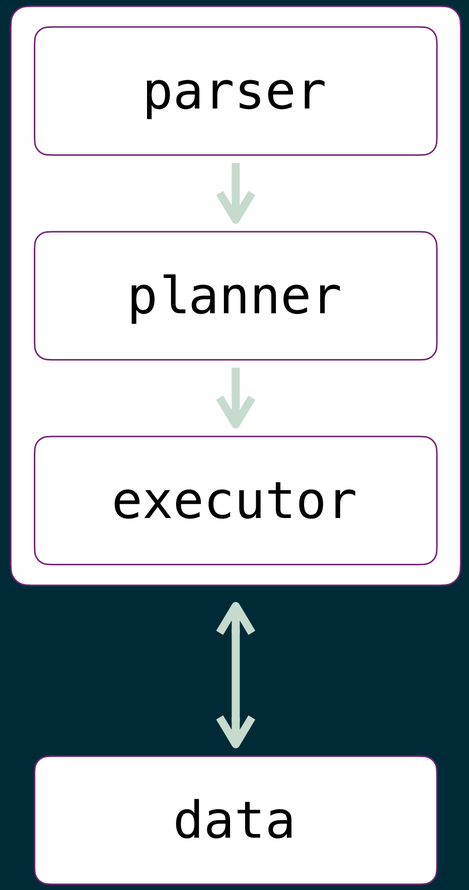
`SELECT lorem FROM ipsum;`

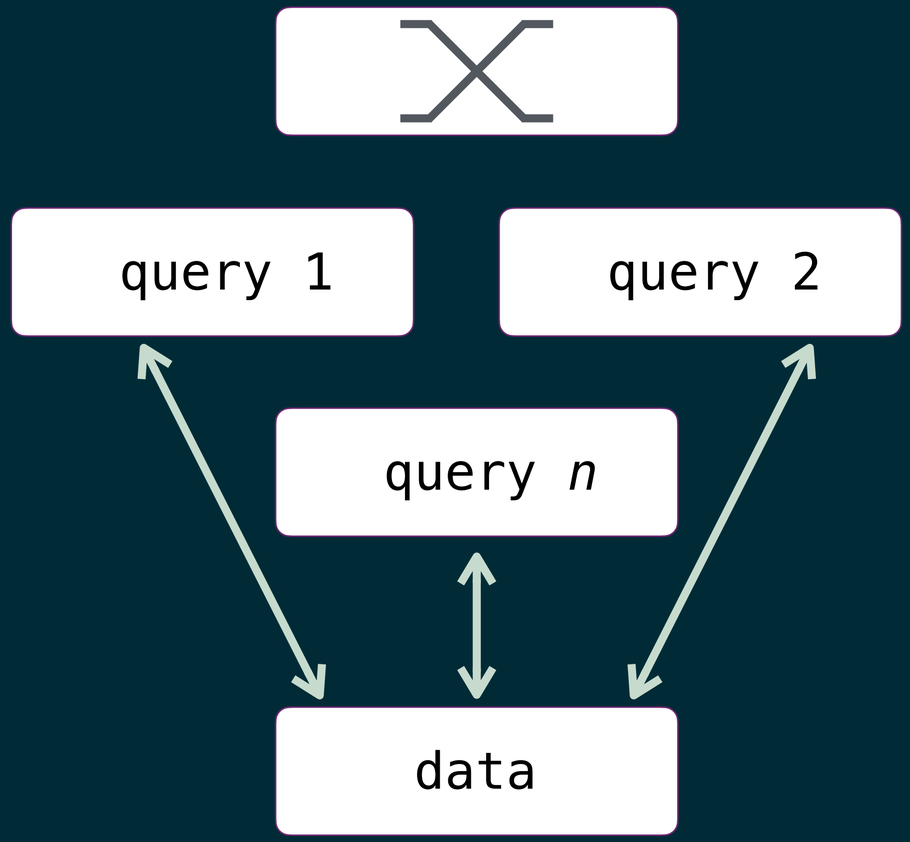
Custom optimal program per syntactically valid SQL query. Each program can only execute one query

Time efficient

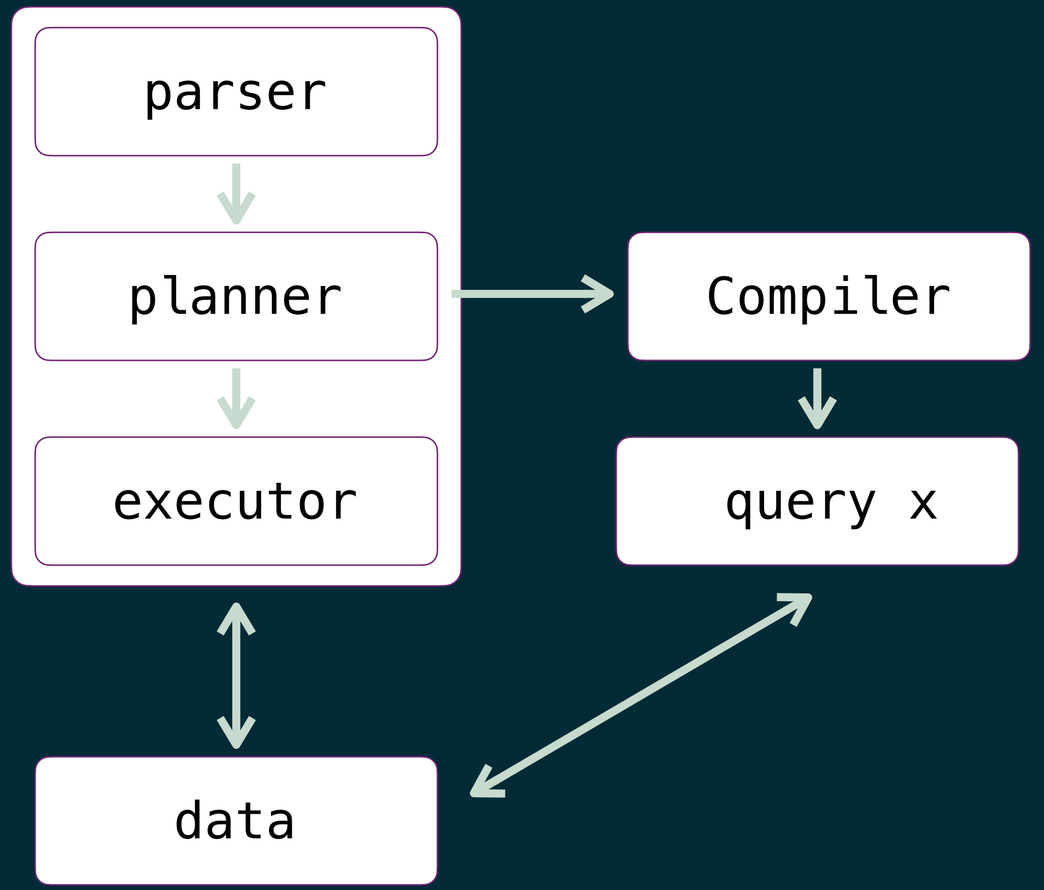


SELECT lorem FROM ipsum;





SELECT lorem FROM ipsum;

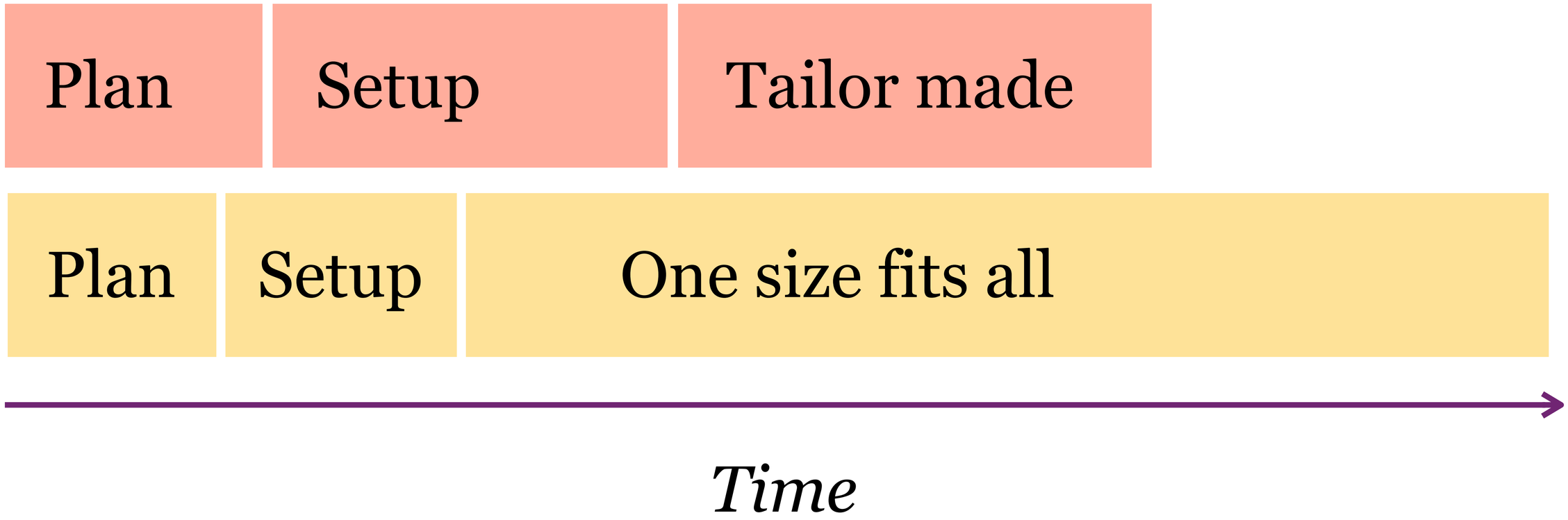


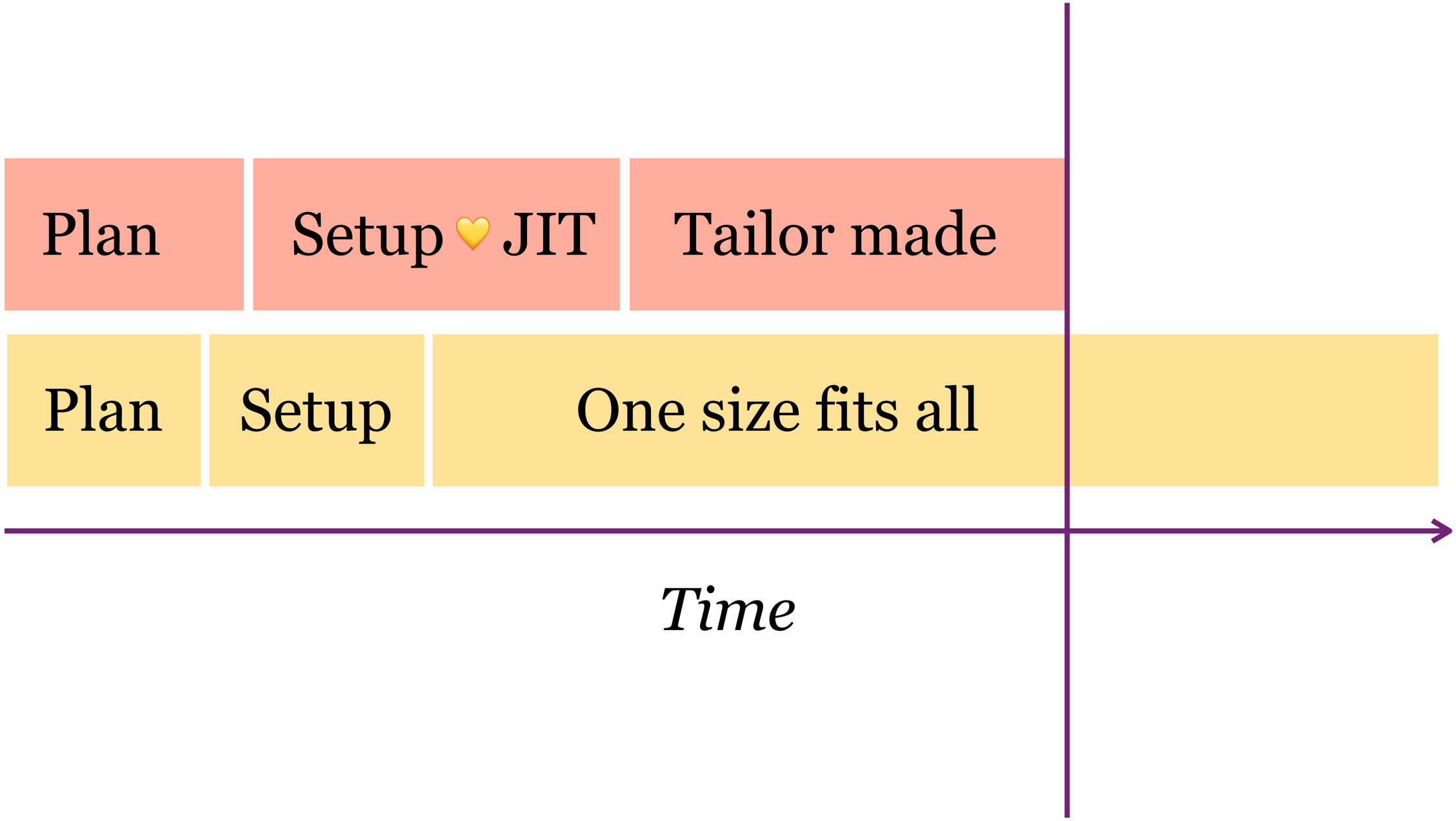
Just In Time is tradeoff management

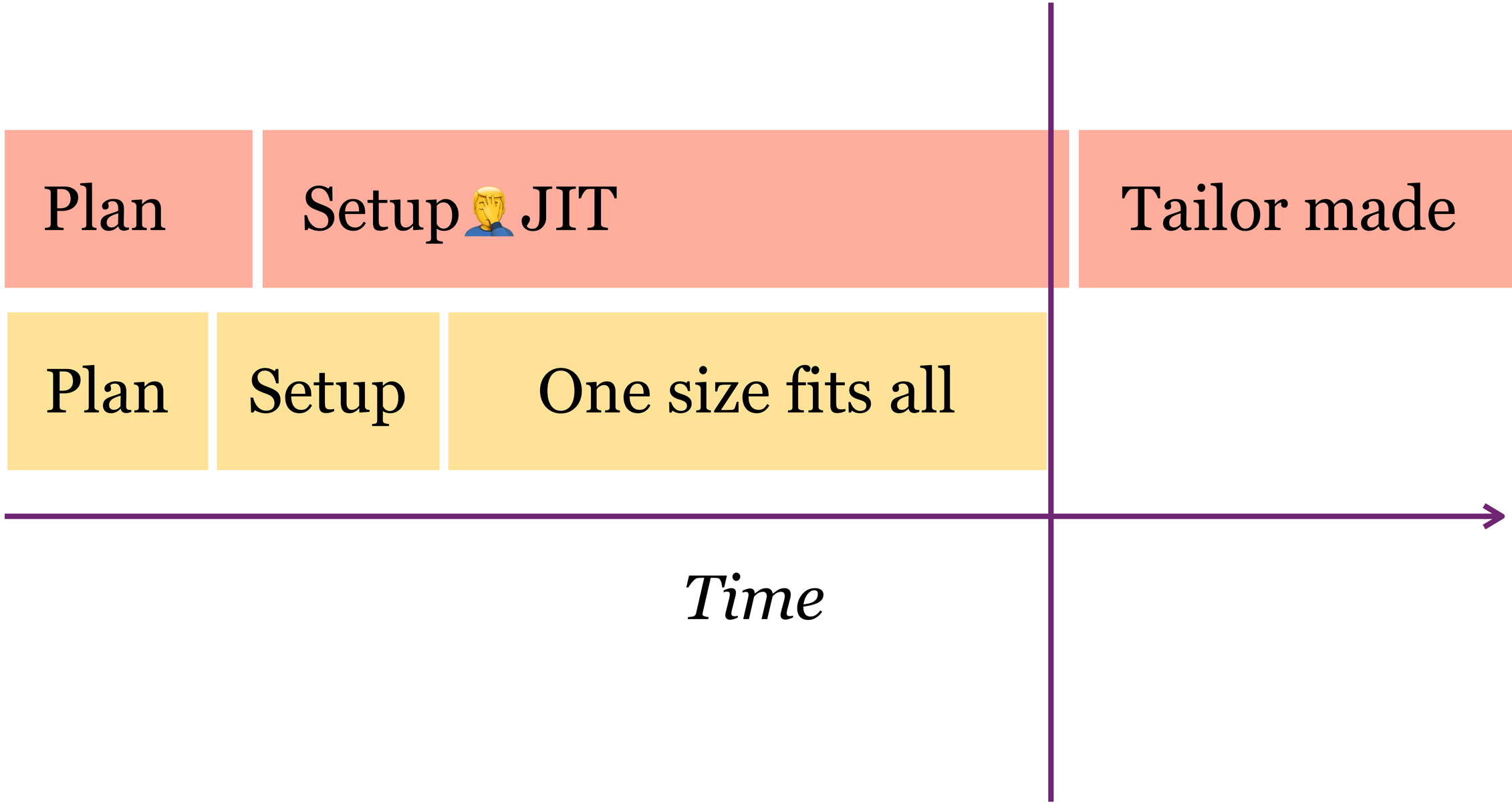
Size of executable

Speed of execution

Altering program characteristics and optimizations based on runtime feedback


Plan
Setup
Tailor made
Plan
Setup
One size fits all
Time


Plan
Setup 💛 JIT
Tailor made
Plan
Setup
One size fits all
Time


Plan
Setup 🤦 JIT
Tailor made
Plan
Setup
One size fits all
Time

JIT IN POSTGRES

| ˌhɒt pəˈteɪtəʊ |

What does it do, what can it do
and what should it do in order
to work better ?


POSTGRES
0
2
8
10
11
GOES TO
ELEVEN

Introduced in Postgres 11 and enabled by default in Postgres 12

Tuple deforming

Inlining

Expression evaluation

Will be disabled by default in Postgres 19

Requires autoconf, meson support is worked on

Tuple deformation is the process of reading a serialized tuple from storage into a memory structure

Knowing the number of columns and their data types can reduce the number of branching needed

Very wide tables in short running queries can however yield surprisingly large generated code which coupled with optimization can get expensive

Expression evaluation is `WHERE` *clauses, target lists, aggregates and projections*

Can yield dramatic speedups for expression-heavy analytical queries

Removing lookups and indirection in function calls

Inlining replace function calls with their compiled representations

The extensibility of Postgres is fantastic during development, but it is paid for at runtime

Inlining can reduce the overhead of the extensible nature

Functions are compiled to IR and the bitcode is installed as `.bc` *files*

```
jit = boolean

jit_provider = string

jit_above_cost = int

jit_inline_above_cost = int

jit_optimize_above_cost = int
```

```
jit = boolean

jit_provider = string

jit_above_cost = int

jit_inline_above_cost = int

jit_optimize_above_cost = int
```

`jit_provider = string`

The JIT compiler backend can be replaced with a custom module, this is PostgreSQL after all

Only llvmjit available at this point, implementing LLVM as a JIT compiler

30.4.2. Pluggable JIT Providers

PostgreSQL provides a JIT implementation based on LLVM. The interface to the JIT provider is pluggable and the provider can be changed without recompiling (although currently, the build process only provides inlining support data for LLVM). The active provider is chosen via the setting **jit_provider**.

30.4.2.1. JIT Provider Interface

A JIT provider is loaded by dynamically loading the named shared library. The normal library search path is used to locate the library. To provide the required JIT provider callbacks and to indicate that the library is actually a JIT provider, it needs to provide a C function named `_PG_jit_provider_init`. This function is passed a struct that needs to be filled with the callback function pointers for individual actions:

```
struct JitProviderCallbacks
{
    JitProviderResetAfterErrorCB reset_after_error;
    JitProviderReleaseContextCB release_context;
    JitProviderCompileExprCB compile_expr;
};

extern void _PG_jit_provider_init(JitProviderCallbacks *cb);
```

How to draw an owl



Fig 1. Draw two circles



Fig 2. Draw the rest of the damn owl

```
struct JitProviderCallbacks
{
    JitProviderResetAfterErrorCB reset_after_error;
    JitProviderReleaseContextCB release_context;
    JitProviderCompileExprCB compile_expr;
};
```

```
struct JitProviderCallbacks
{
    JitProviderResetAfterErrorCB reset_after_error;
    JitProviderReleaseContextCB release_context;
    JitProviderCompileExprCB compile_expr;
};
```

Called in the main Postgres processing loop when an error has been thrown

Reset provider specific error handling

```
struct JitProviderCallbacks
{
    JitProviderResetAfterErrorCB reset_after_error;
    JitProviderReleaseContextCB release_context;
    JitProviderCompileExprCB compile_expr;
};
```

Release resources associated with an Executor state

Called during `FreeExecutorState()`

```
struct JitProviderCallbacks
{
    JitProviderResetAfterErrorCB reset_after_error;
    JitProviderReleaseContextCB release_context;
    JitProviderCompileExprCB compile_expr;
};
```

Compile the expression in the passed `ExprState`

`state->evalfunc`

`state->evalfunc_private`

```
PG_MODULE_MAGIC_EXT(.name = "byojit", .version = PG_VERSION);

void byo_reset_after_error(void) { .. }

void byo_release_context(JitContext *context) { .. }

bool byo_compile_expr(ExprState *state)
{
     ByoExprState *bstate = palloc(..);
     /* Small matter of programming.. */
     state->evalfunc = ExecByoExpr;
     state->evalfunc_private = bstate;
}

void
_PG_jit_provider_init(JitProviderCallbacks *cb)
{
    cb->reset_after_error = byo_reset_after_error;
    cb->release_context = byo_release_context;
    cb->compile_expr = byo_compile_expr;
}
```

`jit_tuple_deforming` *and* `jit_expressions` *are boolean switches for capabilities*

`jit_debugging_support` *allows attaching with GDB if the LLVM used supports it*

`jit_dump_bitcode` *writes out the generated IR code for JIT hacking debugging to* `$data_dir`

`jit_profiling_support` *writes out the data needed to perf profile the generated code to* `~/.debug/jit`



EXTENSIONS

| ɛkˈstɛnʃn |

The very lifeblood of Postgres,
as important to the community
as pizza is for the Ninja Turtles

Function bodies of type C and internal can be inlined, as well as operators based on such functions

When building with PGXS against an installation with JIT enabled, this will be done automatically

JIT = OFF

| WAI |

Ja visst gör det ont när knoppar brister.
Varför skulle annars våren tveka?
Varför skulle all vår heta längtan
bindas i det frusna bitterbleka?
Höljet var ju knoppen hela vintern.
Vad är det för nytt, som tär och spränger?
Ja visst gör det ont när knoppar brister,
ont för det som växer och det som stänger.

> *People are unlikely to notice/report cases when the JIT (including costing) works fine. But they sure are annoyed when it makes the wrong choice.*
>
> *- Tomas Vondra (pgsql-hackers@, 230624)*



James Coleman

Stampede of the JIT compilers

To: PostgreSQL Hackers, Cc: David Pirotte

<CAAaqYe-g-Q0Mm5H9QLcu8cHeMwok+HaxS4-UC9Oj3bK

Hello,

We recently brought online a new database cluster, and in th
of ramping up traffic to it encountered a situation where a
query (analyzing helped with this, but I think the issue is
relevant) resulted in that query being compiled with JIT, an
large number of backends were running that same shape of que
them JIT compiling it. Since each JIT compilation took ~2s,

JC **James Coleman**

Stampede of the JIT

KK **Konstantin Knizhnik**

One more problem with JIT

To: PostgreSQL Hackers

<b0900d21-594a-57a7-d345-9f8e3987e068@garret.ru>

Hi hackers,

I am using pg_embedding extension for Postgres which implement

query looks something like this:

openai <=> ARRAY[0.024

JC

James Coleman

S

KK

Kons

One

To:

<bC

Hi hacke

I am u

AH

Alvaro Herrera

JIT doing duplicative optimization?

To: Pg Hackers

<202111112002.oqjact5562h5@alvherre.pgsql>

```
Hello

Recently I noticed extremely suspicious behavior from queries
A query (details for which sadly I cannot disclose) had an EXP
looking like this:

JIT:
  Functions: 743
  Options: Inlining true, Optimi
```

JC

James Coleman

S

KK

Kons

AH

Alvaro Herrera

JIT doing duplicative optimization?

To: Pg Hackers

PG Bug reporting form

BUG #16076: JIT causes huge delays in a complex query. jit=off solves it.

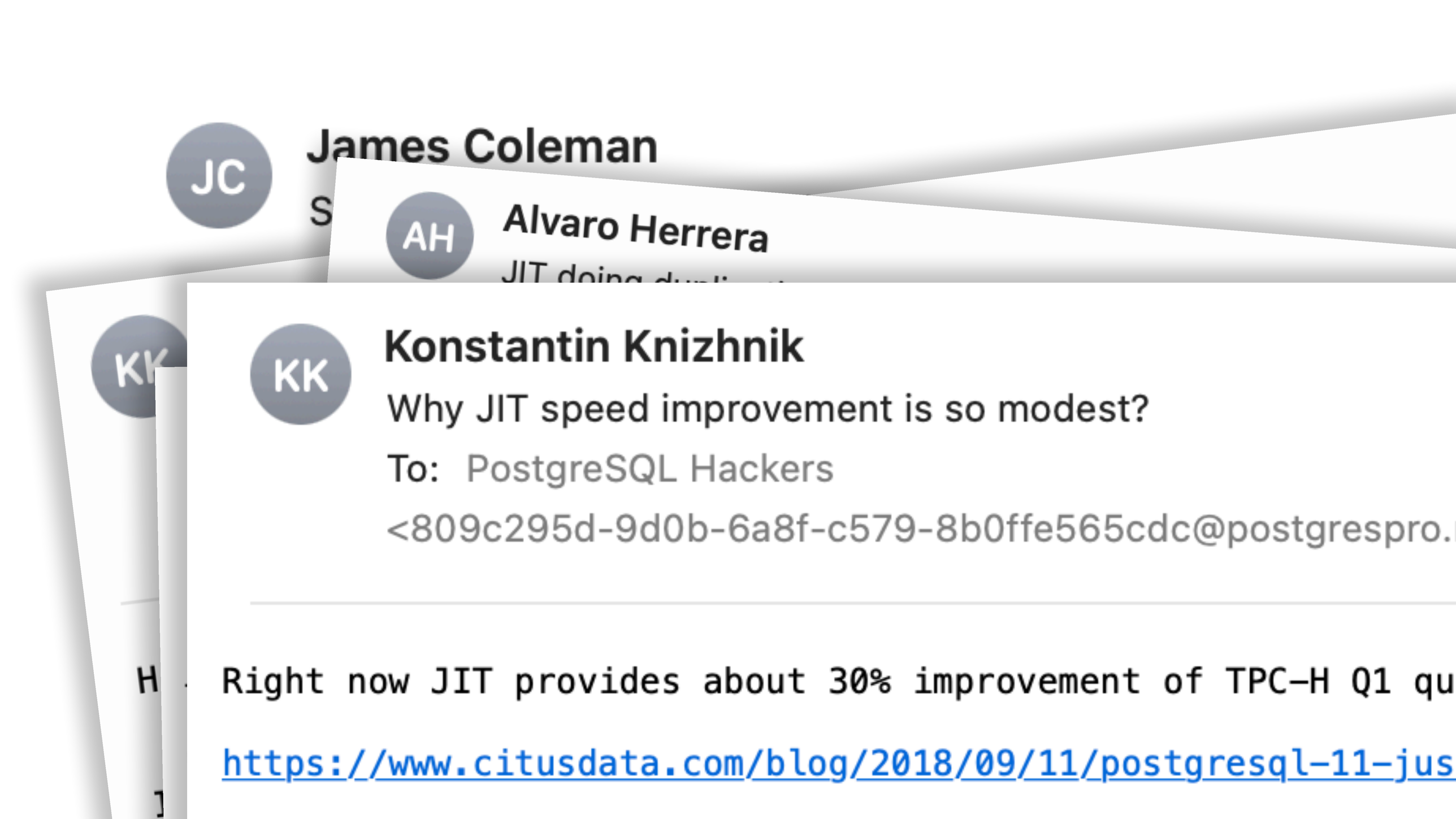
To: pgsql-bugs, Cc: yuriastrakhan@gmail.com

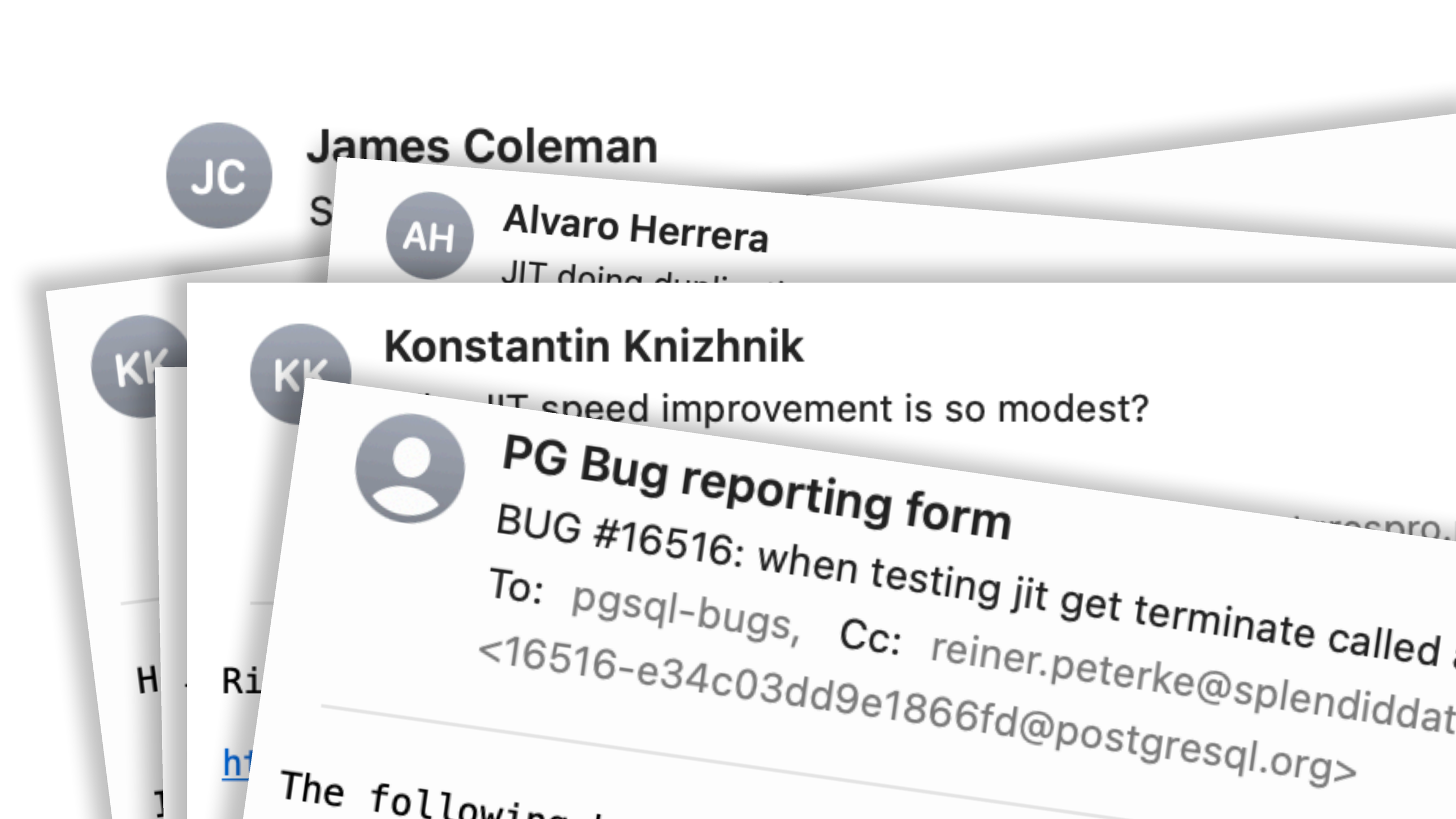
<16076-3ec3880ea1914088@postgresql.org>

H

```
The following bug has been logged on the website:

Bug reference:      16076
Logged by:          Yuri Astrakhan
```


JC
James Coleman
AH
Alvaro Herrera
KK
Konstantin Knizhnik
Why JIT speed improvement is so modest?
To: PostgreSQL Hackers
<809c295d-9d0b-6a8f-c579-8b0ffe565cdc@postgrespro.
Right now JIT provides about 30% improvement of TPC-H Q1 qu
https://www.citusdata.com/blog/2018/09/11/postgresql-11-jus


JC
James Coleman
AH
Alvaro Herrera
KK
Konstantin Knizhnik
speed improvement is so modest?
PG Bug reporting form
BUG #16516: when testing jit get terminate called
To: pgsql-bugs, Cc: reiner.peterke@splendiddat
<16516-e34c03dd9e1866fd@postgresql.org>
The

More and better optimizations?

Clever LLVM tricks?

Tweaked IR code generation?

A new JIT backend?

Yes to some of the above, but the root cause to solve is in Postgres

Crashes with no apparent reproducer or single root cause query

Due to memory "leak" in LLVM during inlining

Types created in LLVM survive for reuse but every query will recreate structurally equivalent types

LLVM memory context reset after n queries which drops the "leaked" types

Crashes with no apparent reproducer or single root cause query

Due to memory "leak" in LLVM during inlining

Types created in LLVM survive for reuse but every query will recreate structurally equivalent types

LLVM memory context reset after n queries which drops the "leaked" types



Crashes with no apparent reproducer or single root cause query

Due to memory "leak" in LLVM during inlining

Types created in LLVM survive for reuse but every query will regenerate structurally equivalent types

LLVM memory context reset after n queries which drops the "leaked" types

FIXED (ish)

Unpredictable invocation

Queries which have no benefit of JIT are compiled with time wasted

Long running analytics queries which could benefit from JIT expression evaluation aren't always compiled

Improving the DML can make queries worse

Coarse costing model

Inefficiences and lack of amortized cost

Often executed queries should be able to benefit from earlier executions

Lack of caching

COSTING

| kɒstɪŋ |

Cash rules everything around
me C.R.E.A.M, get the money
Dollar dollar bill, y'all

```
jit_above_cost = int

jit_inline_above_cost = int

jit_optimize_above_cost = int
```

`jit_above_cost = n`

All queries with a cost higher than `n` get JIT compiled without optimizations applied

Compiled with `-O0`

Default set to `100000`

`-1` *disables JIT compilation*

`jit_optimize_above_cost = n`

All queries with a cost higher than `n` *get JIT compiled with expensive optimizations (*`-O3`*)*

Default set to `500000`

`-1` *disable optimizations*

Recommended to be same as `jit_inline_above_cost`

`jit_optimize_above_cost = n`

All queries with a cost higher than `n` *get JIT compiled with* ***expensive*** *optimizations (*`-O3`*)*

Default set to `500000`

`-1` *disable optimizations*

Recommended to be same as `jit_inline_above_cost`

`jit_inline_above_cost = n`

Inlining will be performed for queries with a cost higher than `n`

Default set to `500000`

`-1` *disables inlining*





> *"JIT costing is disconnected from reality. It's a bit like trying to tune your radio with the volume control."*

- David Rowley (pgsql-hackers@, 230624)

Plan cost does not reflect potential JIT value

A very expensive plan can be IO bound and not benefit from JIT at all

Partitioning can yield very large plantrees

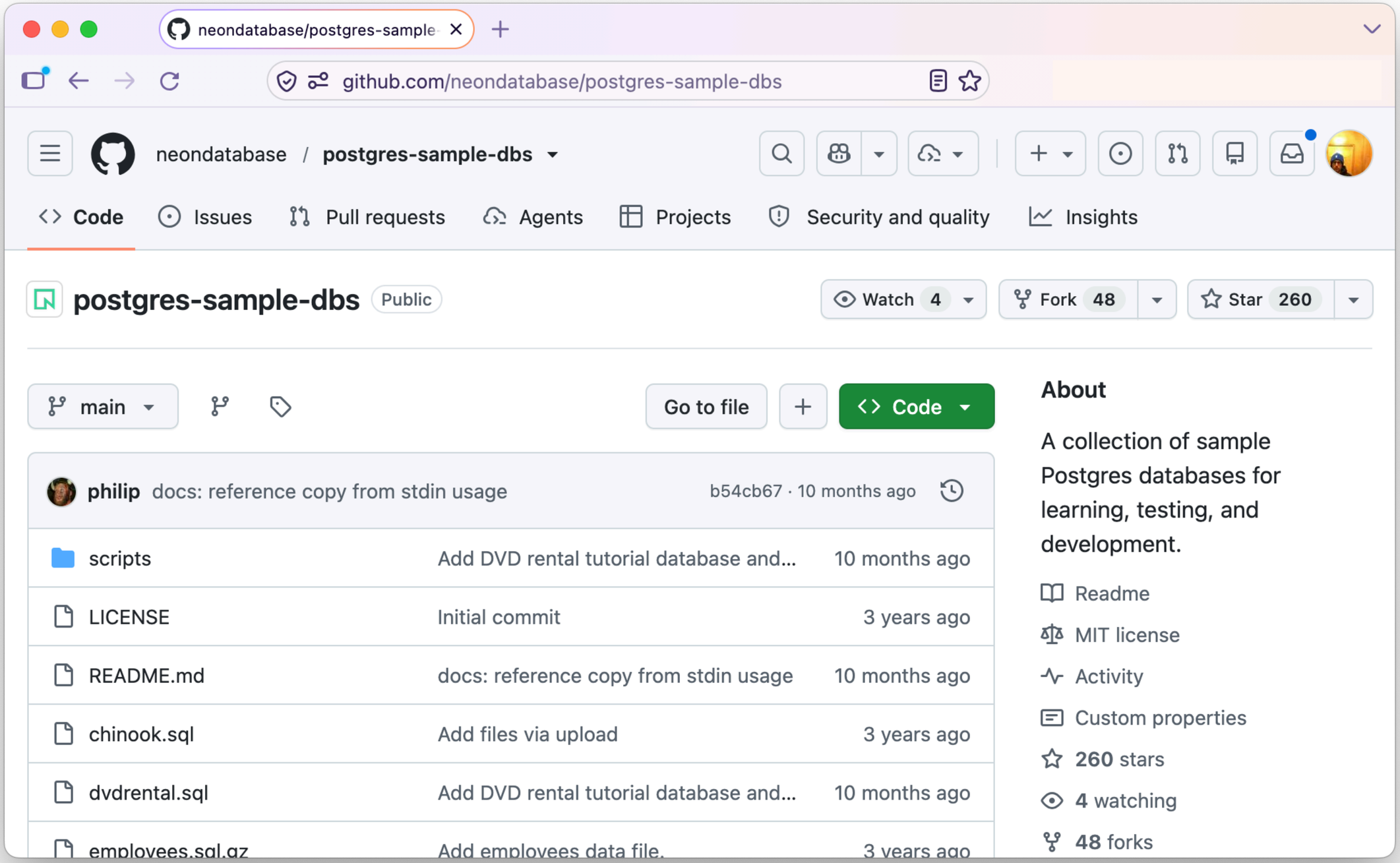
Determine half-way through that JIT is too expensive and back out?

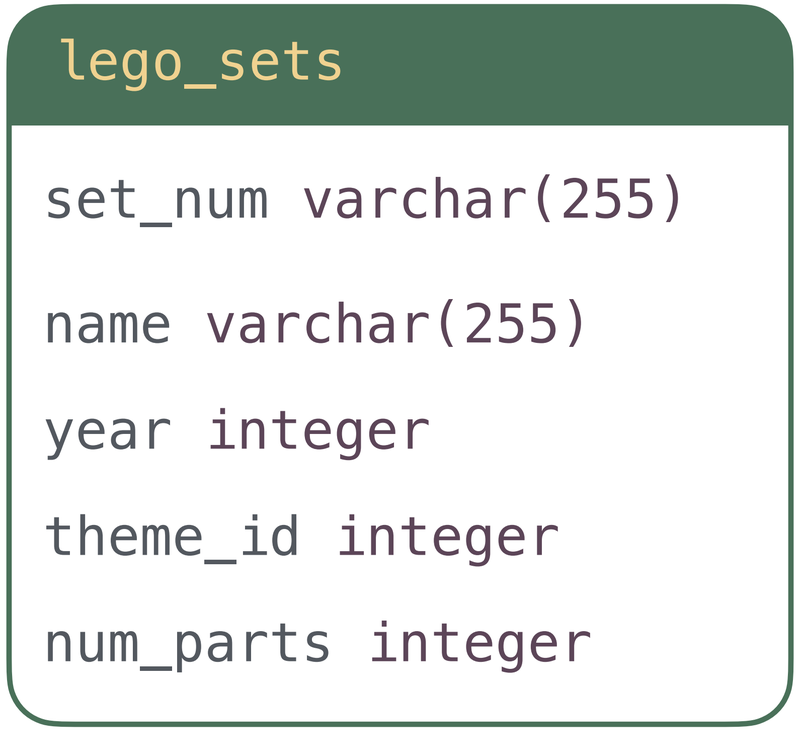
Emitting code is very expensive

Emitting functions one by one is vastly more expensive than emitting all of them at once

> *"It wasn't quite as bad back when it was written - partitioning blows the problem up by an order of magnitude or three. .*

- Andres Freund (pgsql-hackers@, 220329)


neondatabase/postgres-sample-
github.com/neondatabase/postgres-sample-dbs
neondatabase / postgres-sample-dbs
Code
Issues
Pull requests
Agents
Projects
Security and quality
Insights
postgres-sample-dbs
Public
Watch 4
Fork 48
Star 260
main
Go to file
Code
philip docs: reference copy from stdin usage
b54cb67 · 10 months ago
scripts Add DVD rental tutorial database and... 10 months ago
LICENSE Initial commit 3 years ago
README.md docs: reference copy from stdin usage 10 months ago
chinook.sql Add files via upload 3 years ago
dvdrental.sql Add DVD rental tutorial database and... 10 months ago
employees.sql.gz Add employees data file. 3 years ago
About
A collection of sample Postgres databases for learning, testing, and development.
Readme
MIT license
Activity
Custom properties
260 stars
4 watching
48 forks

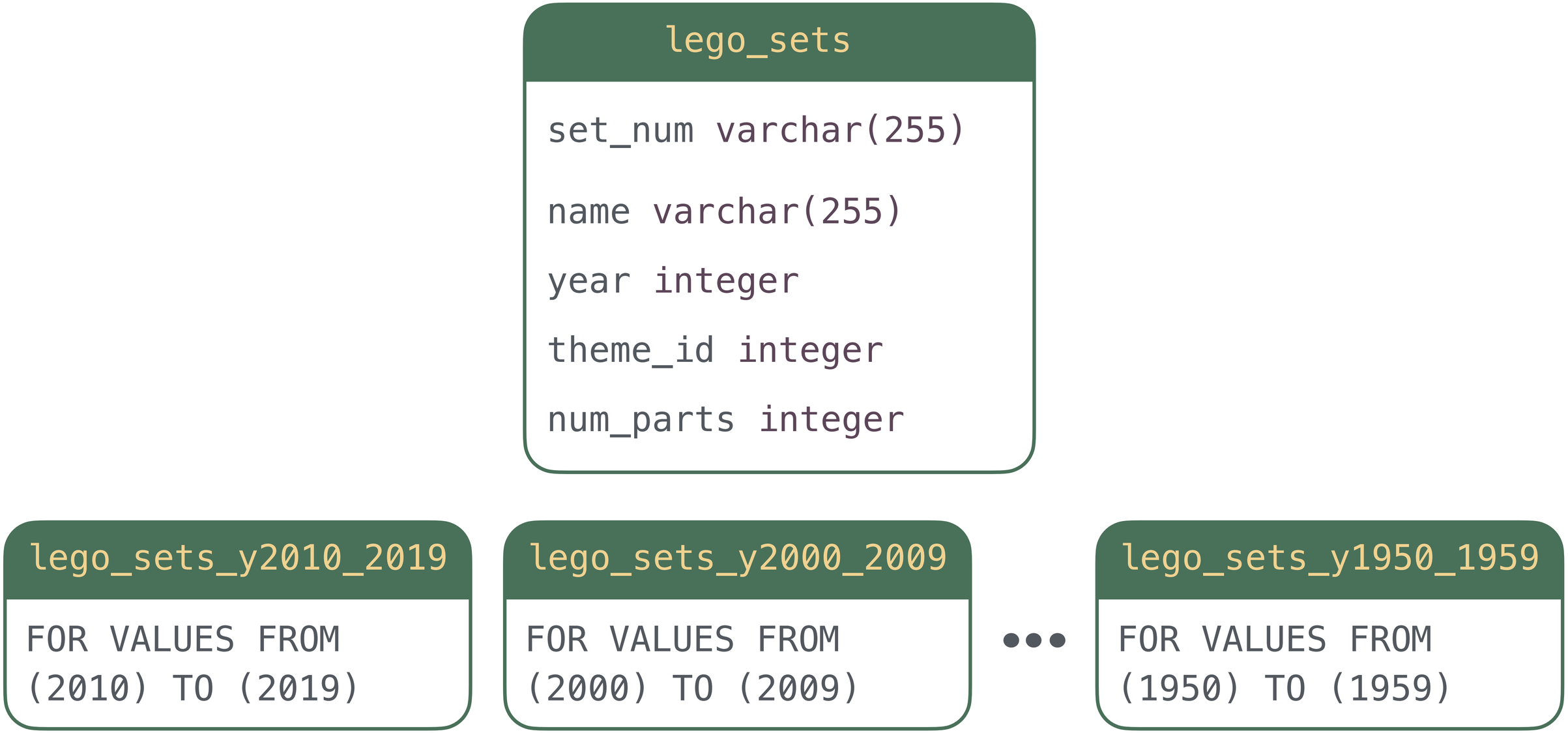

lego_sets
set_num varchar(255)
name varchar(255)
year integer
theme_id integer
num_parts integer

```
EXPLAIN ANALYZE
SELECT
    theme_id,
    percentile_cont(.5)
        within group (order by theme_id) AS median,
    sum(num_parts)
FROM public.lego_sets
   WHERE num_parts BETWEEN 100 AND 2500
   GROUP BY theme_id;
```

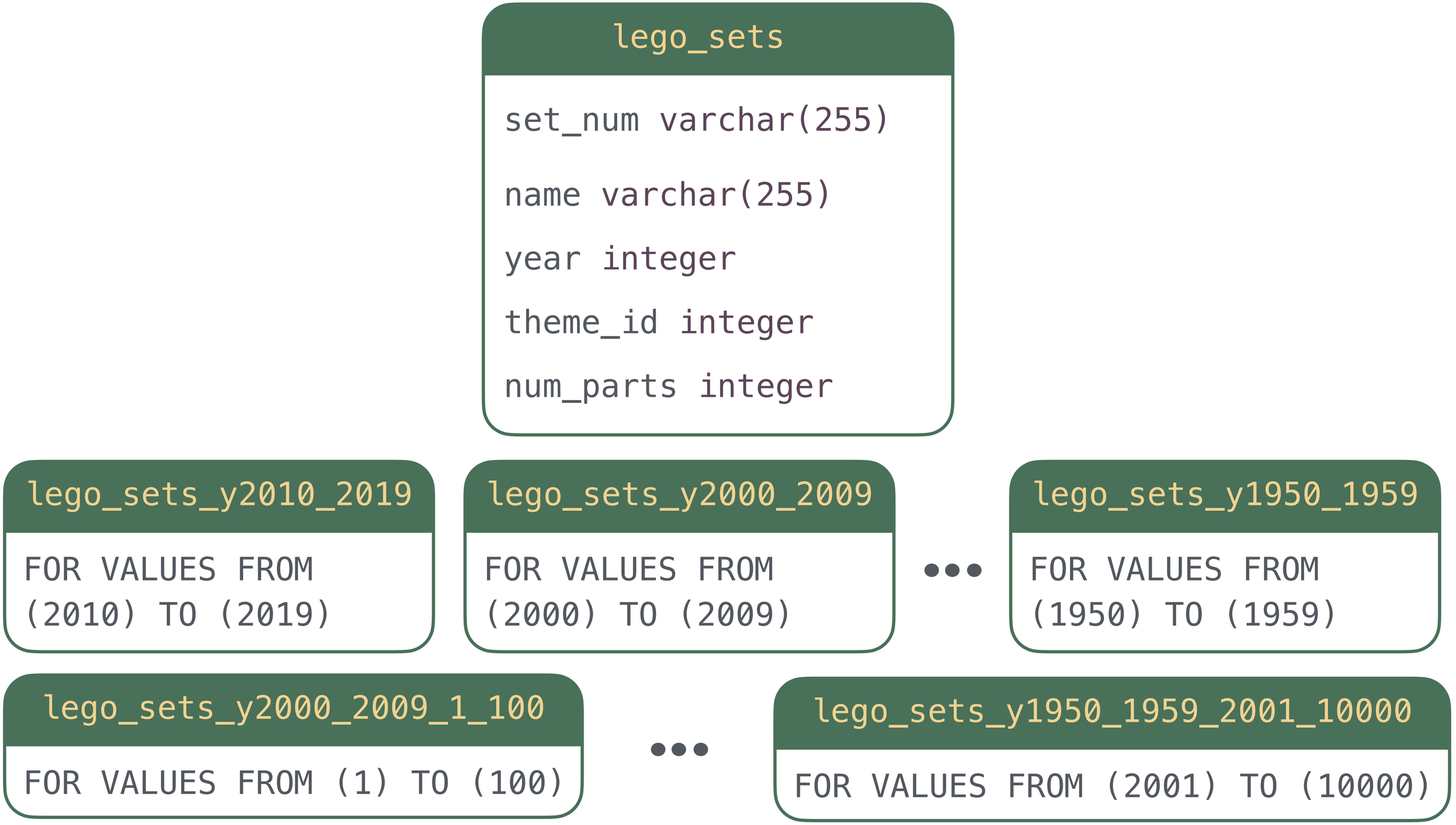
```
                                                
```

```
                                                
```

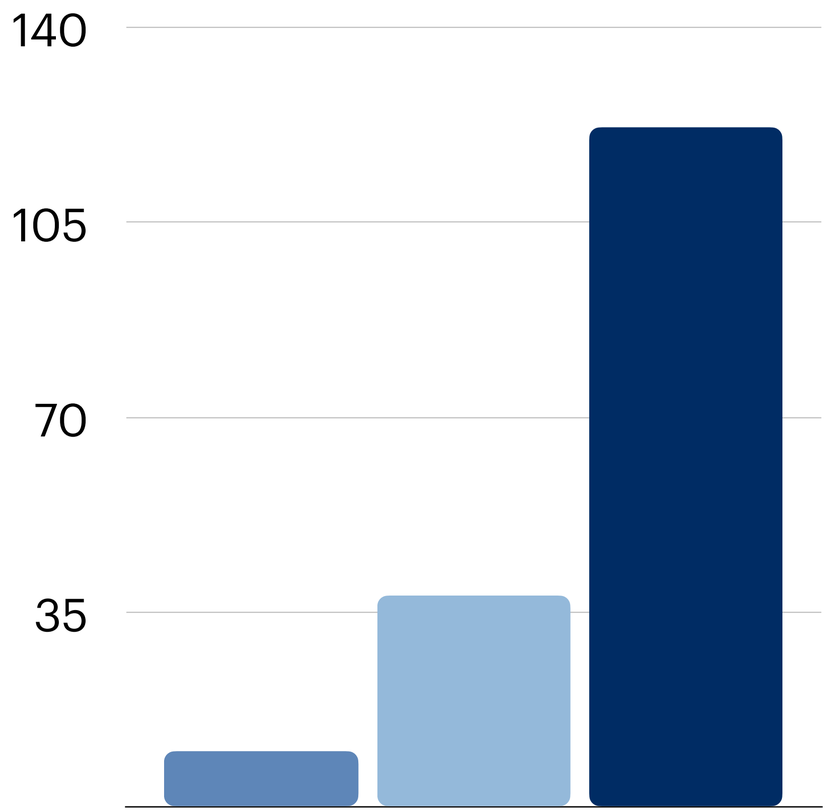
```
Planning Time: 0.358 ms
 JIT:
   Functions: 10
   Options: Inlining true, Optimization true,
   Expressions true, Deforming true
   Timing: Generation 1.634 ms (Deform 0.584 ms),
           Inlining 0.041 ms,
           Optimization 55.891 ms,
           Emission 21.563 ms,
           Total 79.128 ms
 Execution Time: 124.674 ms
(17 rows)
```

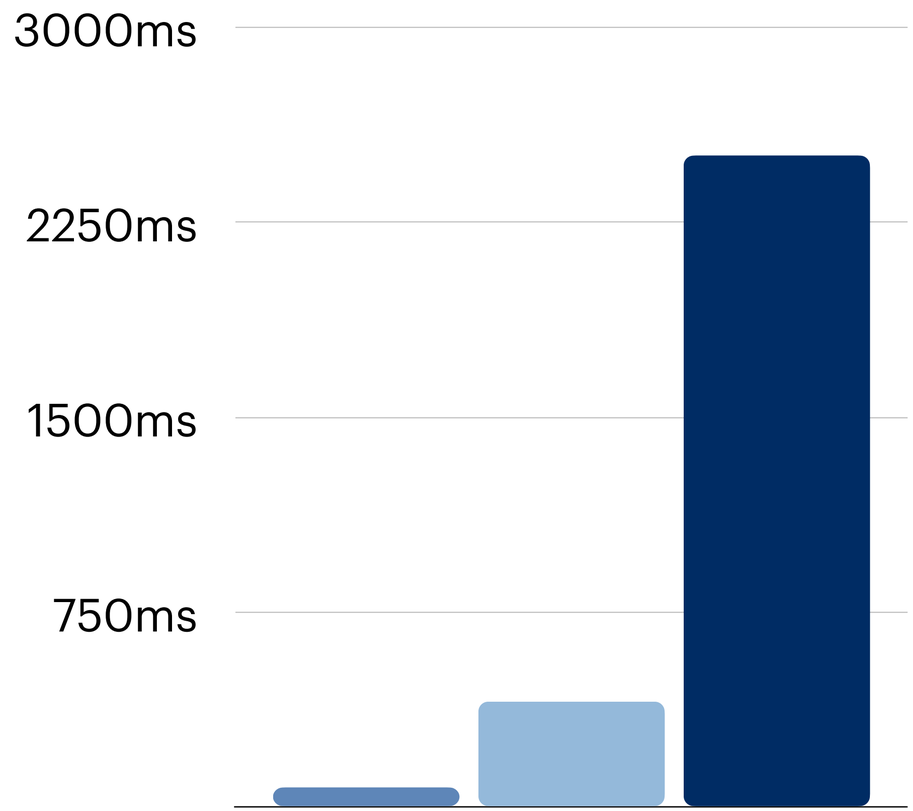

lego_sets
set_num varchar(255)
name varchar(255)
year integer
theme_id integer
num_parts integer
lego_sets_y2010_2019
FOR VALUES FROM (2010) TO (2019)
lego_sets_y2000_2009
FOR VALUES FROM (2000) TO (2009)
lego_sets_y1950_1959
FOR VALUES FROM (1950) TO (1959)

```
 Planning Time: 2.348 ms
 JIT:
   Functions: 38
   Options: Inlining true, Optimization true,
   Expressions true, Deforming true
   Timing: Generation 2.178 ms (Deform 1.128 ms),
           Inlining 0.023 ms,
           Optimization 322.835 ms,
           Emission 80.943 ms,
           Total 405.978 ms
 Execution Time: 431.082 ms
(49 rows)
```


lego_sets
set_num varchar(255)
name varchar(255)
year integer
theme_id integer
num_parts integer
lego_sets_y2010_2019
FOR VALUES FROM (2010) TO (2019)
lego_sets_y2000_2009
FOR VALUES FROM (2000) TO (2009)
lego_sets_y1950_1959
FOR VALUES FROM (1950) TO (1959)
lego_sets_y2000_2009_1_100
FOR VALUES FROM (1) TO (100)
lego_sets_y1950_1959_2001_10000
FOR VALUES FROM (2001) TO (10000)

```
Planning Time: 21.316 ms
 JIT:
   Functions: 122
   Options: Inlining true, Optimization true,
   Expressions true, Deforming true
   Timing: Generation 6.669 ms (Deform 3.543 ms),
           Inlining 0.035 ms,
           Optimization 2232.990 ms,
           Emission 266.489 ms,
           Total 2506.182 ms
 Execution Time: 2543.660 ms
(113 rows)
```





Single

Partitioned

Subpartitioned

Plan time partition pruning helps, narrowing projections with `WHERE` *clauses is a workaround*

Runtime partition pruning still require all functions to be compiled, optimized and emitted only to not be used

Improving granularity of the costing by moving the cost from overall plan cost to per-node cost

Estimated number of executions per node is important

Per sub-tree rather than per plan node

Configuring costs is really hard in general

Not unique to JIT, but excacerbated by it

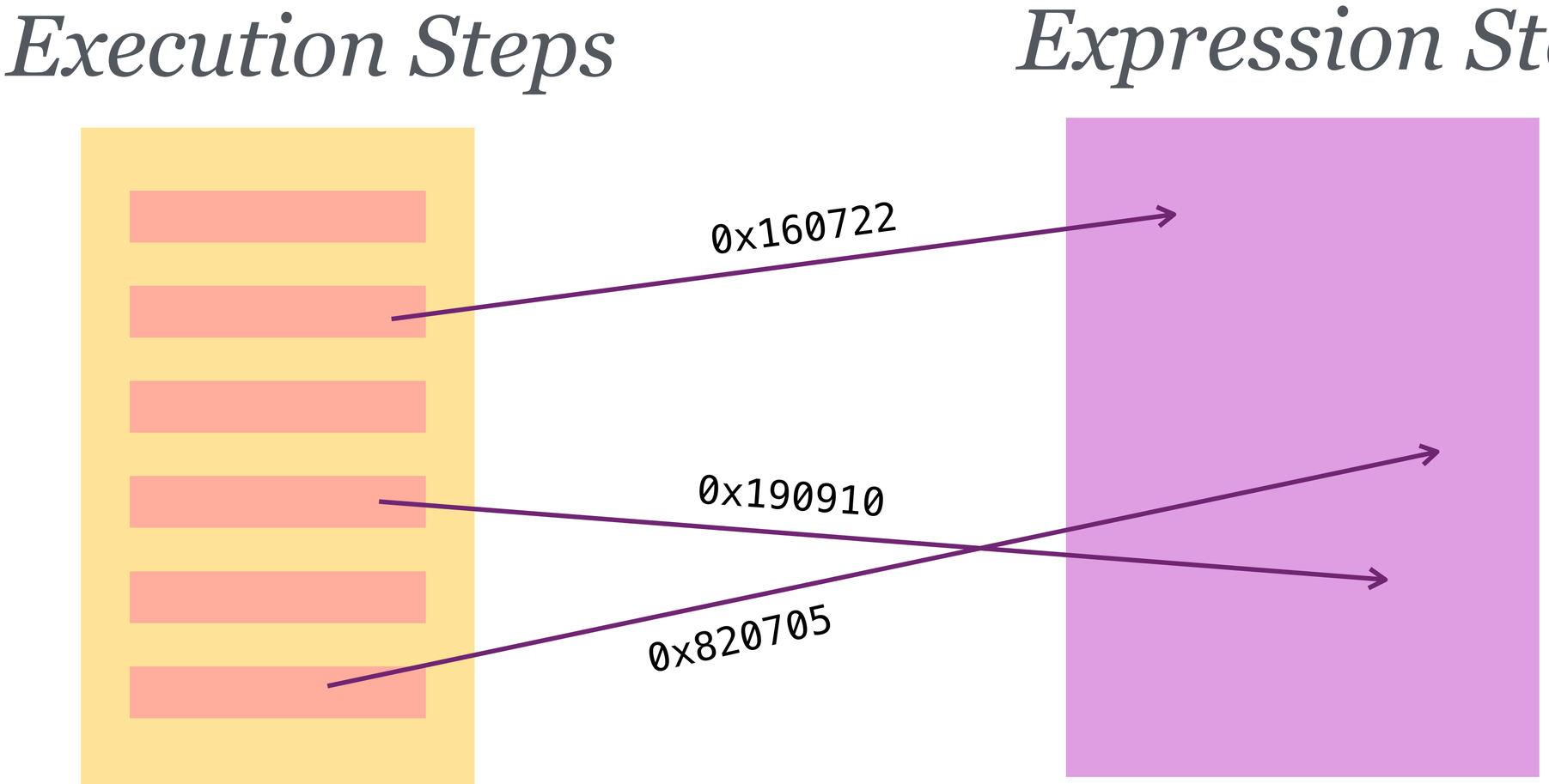


CACHiNG

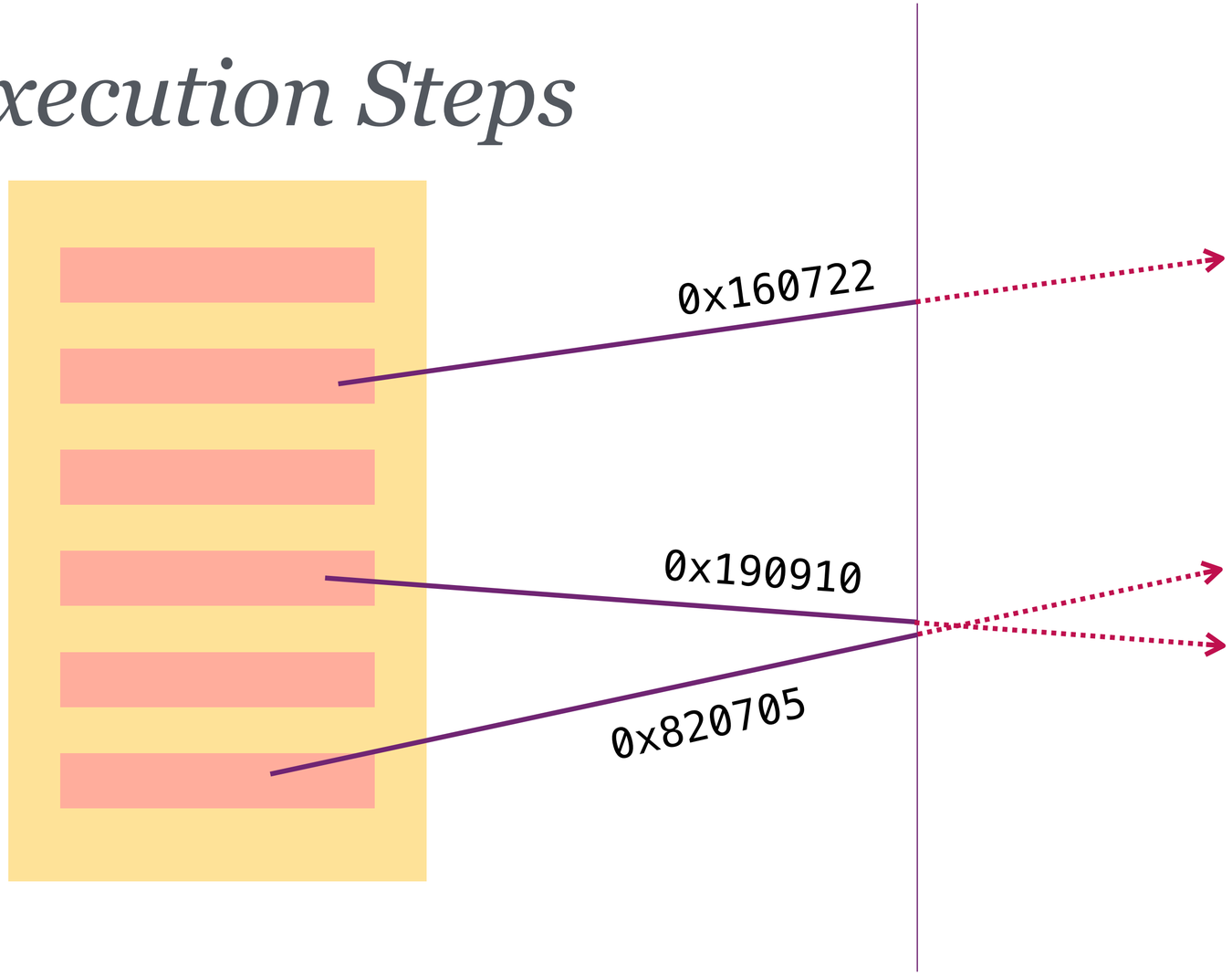
| kaʃɪŋ |

There is only one hard problem in computer science, naming and cache invalidation

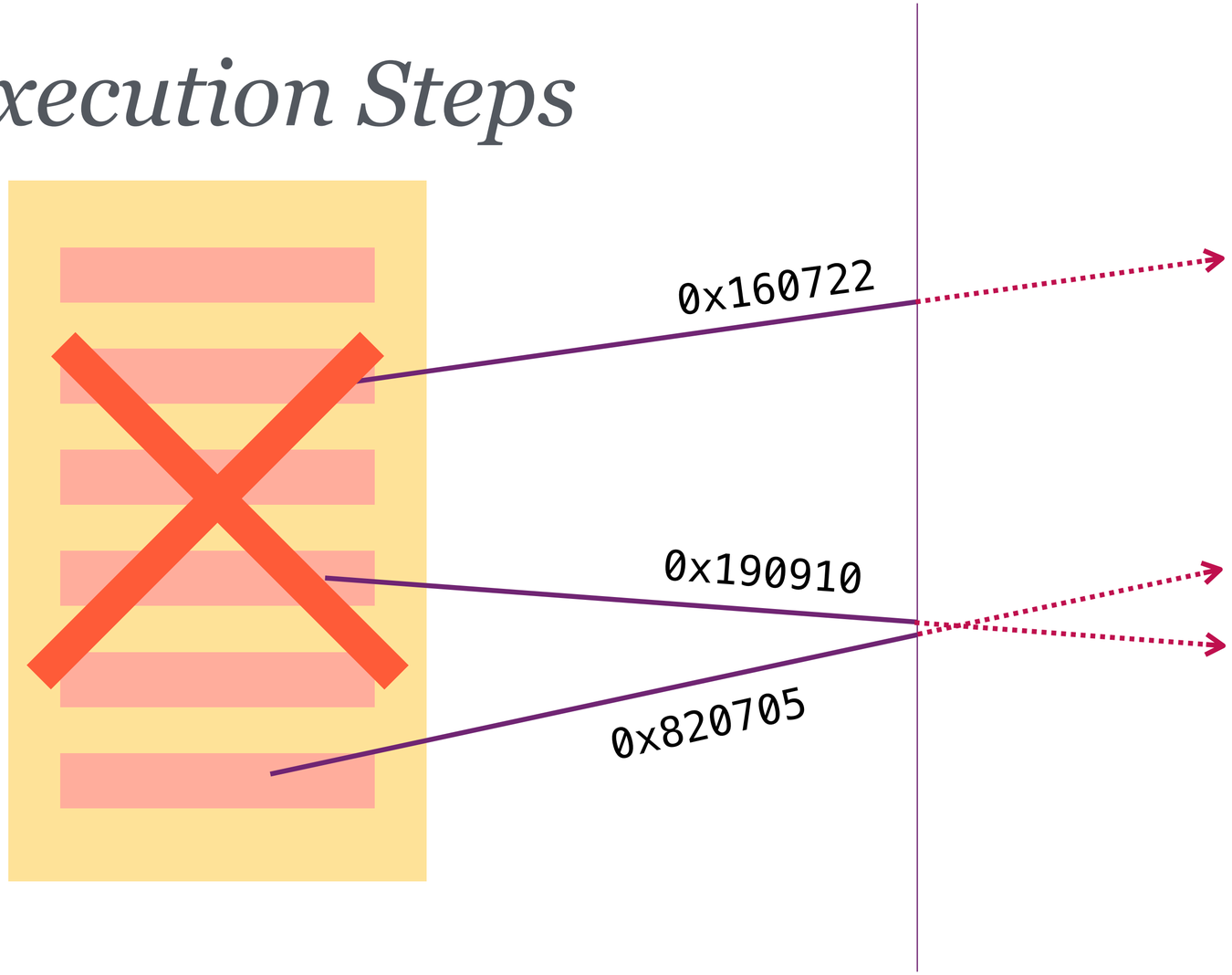
Compiled expressions point to per-query memory allocations


Execution Steps
Expression State
0x160722
0x190910
0x820705

Execution Steps

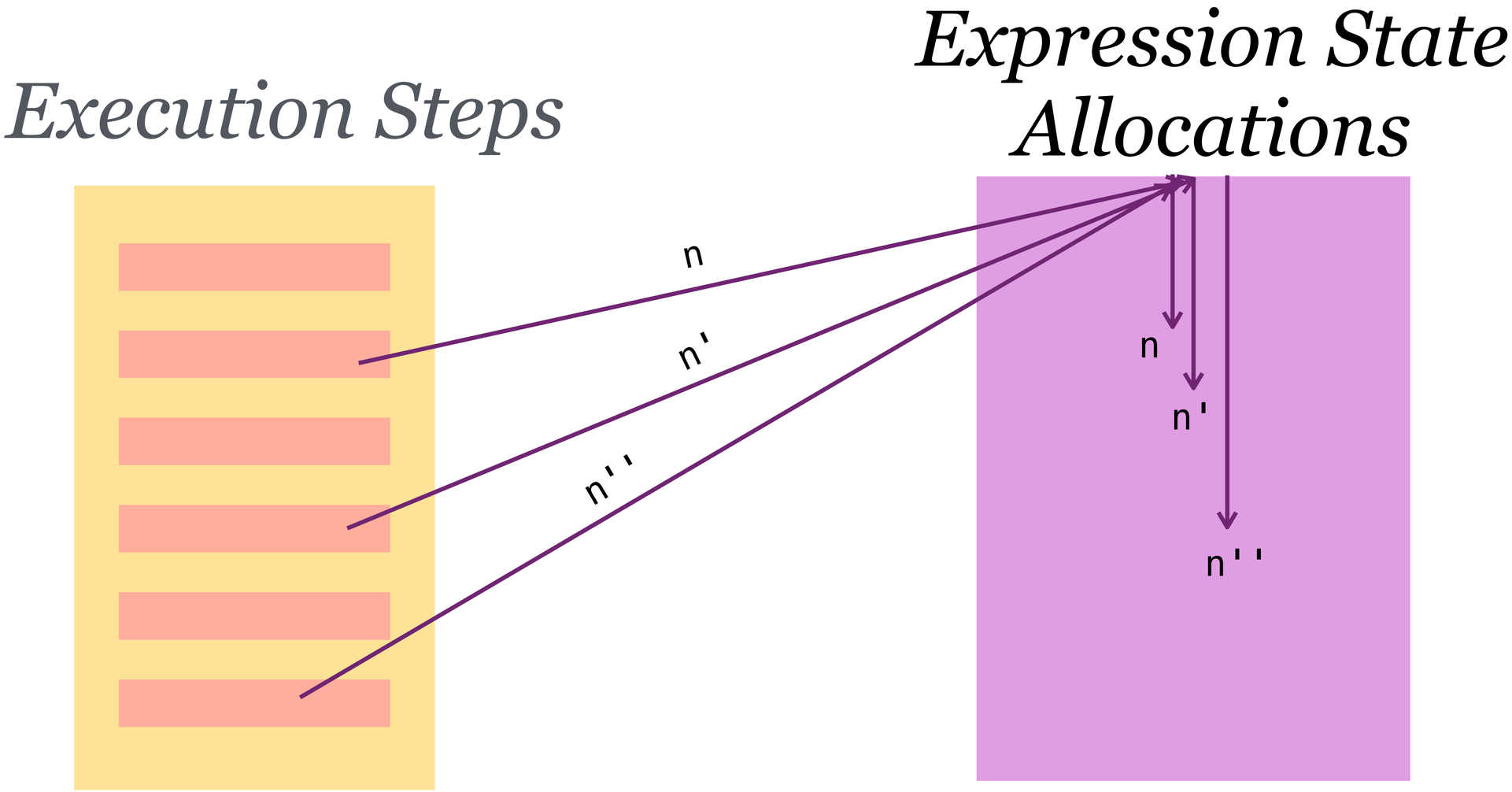


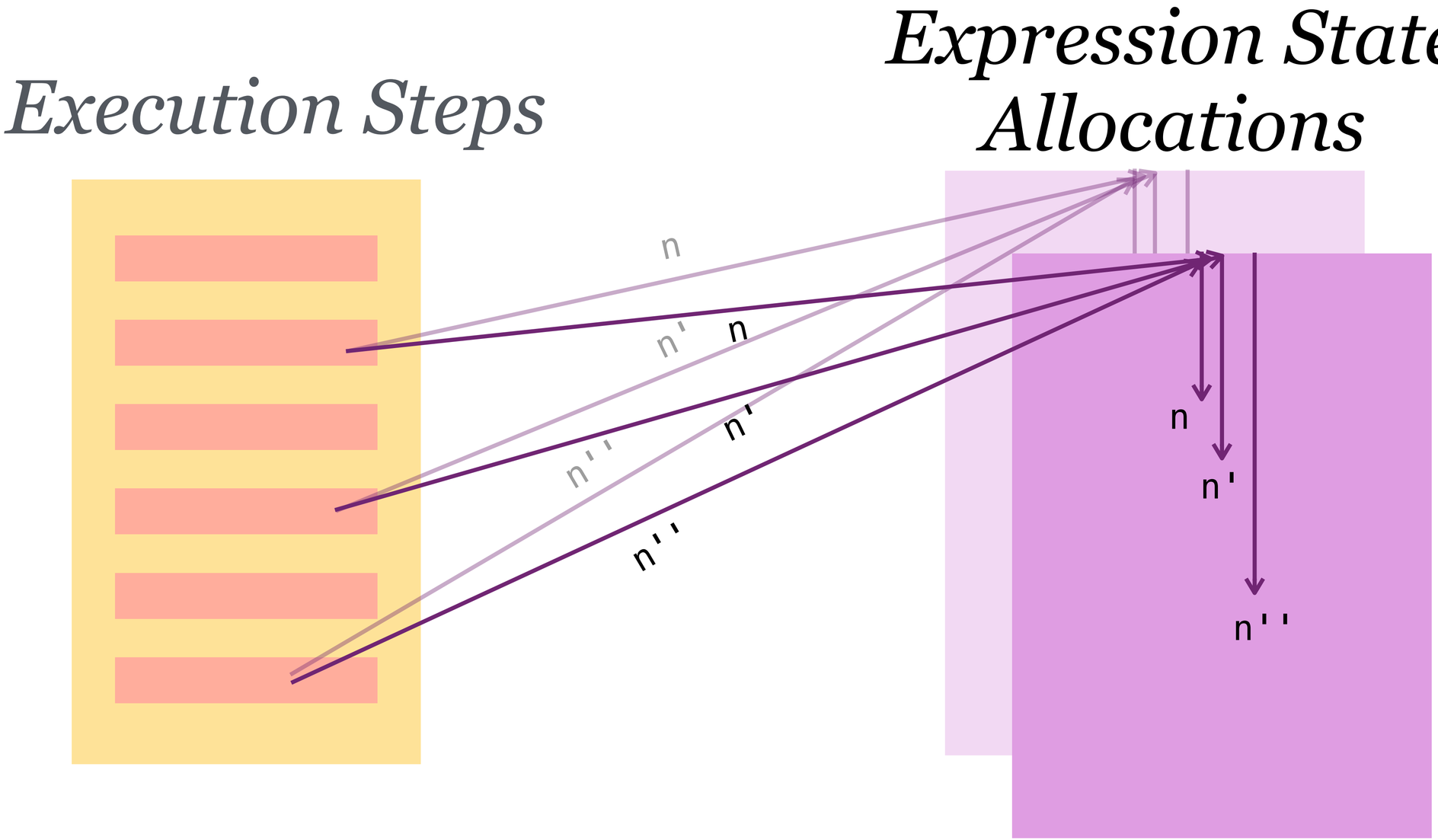
Execution Steps

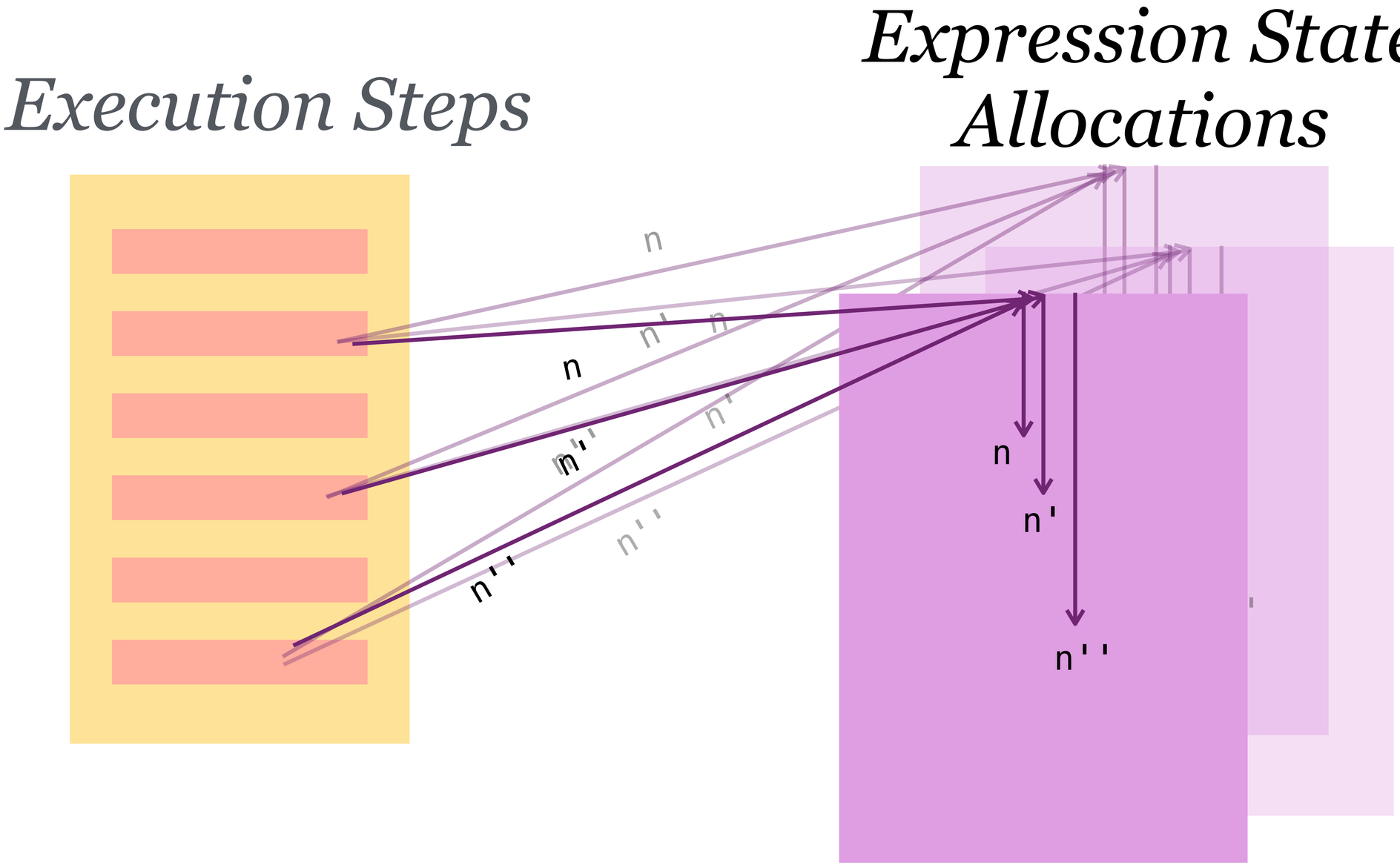


Pointers to expression state must be query agnostic and survive query teardown

Execution steps need to point to the top of the expression state with a base pointer, and use an offset in order to fetch the query specific data


Execution Steps
Expression State Allocations
n
n'
n''
n
n'
n''


Execution Steps
Expression State Allocations
n
n'
n''
n
n'
n''
n
n'
n''


Execution Steps
Expression State Allocations
n
n'
n''

`ExprEvalStep` *cannot have pointers to per-query data*

Basepointer and an offset

Runtime resolution of relative pointers

```
case T_BoolExpr:
{
    BoolExpr   *boolexpr = (BoolExpr *) node;
    int         nargs = list_length(boolexpr->args);
    List       *adjust_jumps = NIL;
    int         off;
    ListCell   *lc;

    /* allocate scratch memory used by all steps of AND/OR */
    if (boolexpr->boolop != NOT_EXPR)
        scratch.d.boolexpr.anynull = palloc_object(bool);
```

```
case T_BoolExpr:
{
    BoolExpr   *boolexpr = (BoolExpr *) node;
    int         nargs = list_length(boolexpr->args);
    List       *adjust_jumps = NIL;
    int         off;
    ListCell   *lc;

    /* allocate scratch memory used by all steps of AND/OR */
    if (boolexpr->boolop != NOT_EXPR)
-       scratch.d.boolexpr.anynull = (bool *) palloc(sizeof(bool));
+       scratch.d.boolexpr.anynull = ExprBuilderAllocBool();
```

```
EEO_CASE(EEOP_BOOL_AND_STEP_FIRST)
{
    *op->d.boolexpr.anynull = false;
}
```

```
 EEO_CASE(EEOP_BOOL_AND_STEP_FIRST)
 {
-    *op->d.boolexpr.anynull = false;
+    bool *anynull =
+        RELPTR_RESOLVE(data, op->d.boolexpr.anynull);
+
+    *anynull = false;
 }
```

```
 /* for EEOP_BOOL_*_STEP */
 struct
 {
-       bool        *anynull;    /* track if any input was NULL */
+       RelBool      anynull;    /* track if any input was NULL */
        int          jumpdone;   /* jump here if result determined */
 } boolexpr;
```

```
typedef struct ExprRelPtr
{
   uint32 off;
   uint32 llvm_metadata;     /* For LLVM to identify allocation */
} ExprRelPtr;

typedef struct RelBool
{
   ExprRelPtr ptr;
} RelBool;
```

```
RelBool
ExprBuilderAllocBool(ExprStateBuilder *esb)
{
    ExprStateAllocation *alloc = palloc(sizeof(ExprStateAllocation));
    esb->allocation_size = MAXALIGN(esb->allocation_size);

    alloc->ptr.off = esb->allocation_size;
    alloc->ptr.allocno = list_length(esb->allocations) + 1;
    alloc->sz = sizeof(NullableDatum);
    alloc->kind = ERP_BOOL;
    alloc->zeroed = false;
    alloc->initial_content = NULL;
    esb->allocation_size += sz;
    esb->allocations = lappend(esb->allocations, alloc);

    return (RelBool){alloc->ptr};
}
```

Just one piece of the puzzle, similar work needed for `FunctionCallInfo->{flinfo, context}` *etc*

Patch in progress, then Postgres 19 happened..

Based on earlier work from Andres

Some of the preparatory work has already landed

Lots of work remains but hopefully more parts will start to land in Postgres 20

Caching can unlock interesting optimizations

Start with `-O0` version and compile with `-O3` in the background, swap code mid-flight when done

Multiple copies with different optimizations for different queries based on heuristics

TL;dR

| snu:z |

Snooze and you lose

JIT won't be on by default in 20

Maybe it will never be on by default (again)

Not really as dramatic as it sounds, many SaaS and hyperscalers had it off already

Very valuable feature with needs a bit of love and attention

Work is in progress

Have you listened to?

TalkingPostgres.com



Microsoft

Thank you!

daniel@yesql.se

dgustafsson@microsoft.com